

1. What is Kubernetes networking and how does it work?

Answer:

Kubernetes networking enables communication between Pods, Services, and external clients.

Key components:

- **Pod-to-Pod Communication:**
Each Pod gets a unique IP address and can communicate with other Pods within the cluster.
 - **Service-to-Pod Communication:**
Since Pod IPs are dynamic, Services provide a **stable endpoint (ClusterIP)** to access Pods.
 - **Ingress Controllers:**
Manage external HTTP/HTTPS traffic and route requests to appropriate Services.
 - **Network Policies:**
Control traffic between Pods and namespaces (e.g., restrict communication between namespaces A and C).
-

Follow-up Questions:

- How does Kubernetes networking differ from traditional networking?
 - What is CNI, and which plugins have you used (Calico, Flannel, Cilium)?
 - How does DNS resolution work inside Kubernetes?
 - What happens if two Pods are on different nodes?
-

2. What is Role-Based Access Control (RBAC) in Kubernetes?

Answer:

RBAC is a security mechanism used to control access to Kubernetes resources.

Components:

- **Role / ClusterRole:** Define permissions (get, list, watch, create, delete).
- **RoleBinding / ClusterRoleBinding:** Assign roles to users or service accounts.

Example:

A “pod-reader” role may allow:

- get
 - list
 - watch on Pods
-

Follow-up Questions:

- Difference between Role and ClusterRole?
 - How do you secure a production cluster using RBAC?
 - How do service accounts interact with RBAC?
 - What is least privilege access in Kubernetes?
-

3. How does Kubernetes autoscaling work?

Answer:

Kubernetes supports multiple autoscaling mechanisms:

- **Horizontal Pod Autoscaler (HPA):**
Scales Pods based on CPU, memory, or custom metrics.
- **Vertical Pod Autoscaler (VPA):**
Adjusts CPU and memory requests/limits of Pods.
- **Cluster Autoscaler:**
Adds/removes nodes based on workload demand.

Tools like **Karpenter** can also be used for dynamic node provisioning.

Follow-up Questions:

- Can HPA and VPA be used together?
 - How does HPA get metrics?
 - What are custom metrics in HPA?
 - Difference between Cluster Autoscaler and Karpenter?
-

4. How do you debug a Kubernetes Pod?

Answer:

Common debugging steps:

- Check logs:

`kubectl logs <pod>`

- Describe Pod:

`kubectl describe pod <pod>`

- Exec into container:

`kubectl exec -it <pod> -- /bin/bash`

- Check events:

`kubectl get events`

- List Pods:

`kubectl get pods`

Follow-up Questions:

- How do you debug CrashLoopBackOff?
 - How do you debug networking issues between Pods?
 - What if logs are not sufficient?
 - How do you debug init container failures?
-

5. How do you perform rolling updates and rollbacks?

Answer:

- **Rolling Updates:**

Kubernetes gradually replaces old Pods with new ones without downtime.

- Trigger update:

`kubectl set image deployment <name> <container>=<image>`

- **Rollback:**

`kubectl rollout undo deployment <name>`

This ensures zero downtime by replacing Pods one by one.

Follow-up Questions:

- What is maxSurge and maxUnavailable?
 - How do you pause/resume a rollout?
 - How do you check rollout status?
 - What happens if a rollout fails?
-

6. What is Ingress and how does it work?

Answer:

Ingress is an API object that manages external HTTP/HTTPS access to services.

- Routes traffic based on:
 - Hostname (e.g., myapp.example.com)
 - Path (e.g., /api)

Flow:

Client → Ingress → Service → Pod

Follow-up Questions:

- Difference between Ingress and LoadBalancer?
 - What is an Ingress Controller?
 - Have you configured NGINX Ingress or ALB Ingress?
 - How do you enable TLS in Ingress?
-

7. What are resource requests and limits?

Answer:

- **Requests:** Minimum CPU/memory guaranteed to a container.

- **Limits:** Maximum CPU/memory a container can use.

Example:

- Request: 256MB
- Limit: 512MB

This ensures fair resource allocation and avoids overuse.

Follow-up Questions:

- What happens if requests are not defined?
 - How does Kubernetes scheduler use requests?
 - What is QoS class (Guaranteed, Burstable, BestEffort)?
 - How do limits impact performance?
-

8. What happens if a Pod exceeds resource limits?

Answer:

- **Memory exceeded:**
Pod is killed with **OOMKilled error**.
 - **CPU exceeded:**
Pod is **throttled**, not killed → application becomes slow.
-

Follow-up Questions:

- How do you monitor OOMKilled issues?
 - How do you prevent memory leaks?
 - What is CPU throttling in Linux cgroups?
 - How do limits affect autoscaling?
-

9. What are Init Containers?

Answer:

Init containers run before the main application container.

Use cases:

- Load configuration
- Fetch secrets
- Wait for dependencies (e.g., database)

Main container starts only after init containers succeed.

Follow-up Questions:

- Difference between init container and sidecar?
 - Can init containers run in parallel?
 - What happens if init container fails?
 - How do you debug init containers?
-

10. How does Kubernetes handle Pod disruptions and high availability?**Answer:**

Kubernetes ensures high availability using:

- **Pod Disruption Budget (PDB):**
Ensures minimum Pods remain available.
- **Pod Affinity / Anti-affinity:**
Controls Pod placement across nodes.
- **Node Selector / Taints & Tolerations:**
Controls scheduling of workloads.

This prevents all Pods from running on a single node.

Follow-up Questions:

- Difference between affinity and anti-affinity?

- What are taints and tolerations with example?
 - How do you design HA for production workloads?
 - What happens during node failure?
-

Interview Tip (Important for you)

Right now, you're at **6–8 years DevOps level**.

Interviewers expect **this extra depth**:

Instead of just answering:

"HPA scales based on CPU"

Say:

"HPA uses metrics-server for resource metrics, but for production we integrate Prometheus Adapter for custom metrics like request rate or queue length."

That's what moves you to **architect-level answers**.

1. What are the core components of Kubernetes?

Answer:

Kubernetes architecture is divided into:

Control Plane (Master Components):

- **API Server:** Entry point for all cluster communication.
- **etcd:** Distributed key-value store holding cluster state.
- **Scheduler:** Assigns Pods to nodes based on resource requirements.
- **Controller Manager:** Maintains desired state (replicas, nodes, endpoints).

Worker Node Components:

- **Kubelet:** Ensures containers are running as expected.
 - **Kube-proxy:** Handles networking and service communication.
-

Follow-up Questions:

- What happens if etcd goes down?
 - How does scheduler decide node placement?
 - Difference between controller manager and scheduler?
 - What is control plane high availability?
-

2. What is a Pod in Kubernetes?

Answer:

A Pod is the **smallest deployable unit** in Kubernetes.

- It can contain **one or more containers**
 - Containers share:
 - Network (same IP)
 - Storage (volumes)
-

Follow-up Questions:

- Can Pods communicate using localhost?
 - When do you use multi-container Pods?
 - What is Pod lifecycle?
 - Difference between Pod and container?
-

3. What is kubectl?

Answer:

kubectl is the CLI tool used to interact with Kubernetes cluster via API Server.

Examples:

```
kubectl get pods
```

```
kubectl get services
```

```
kubectl logs <pod>
```

Follow-up Questions:

- How does kubectl authenticate to cluster?
 - Where is kubeconfig stored?
 - Can you interact without kubectl?
 - What is context in kubeconfig?
-

4. What is a Deployment?

Answer:

A Deployment manages the lifecycle of Pods.

Features:

- Maintains desired replicas
 - Supports rolling updates
 - Supports rollback
 - Self-healing (recreates failed Pods)
-

Follow-up Questions:

- Difference between Deployment and ReplicaSet?
 - When to use StatefulSet instead?
 - What is revision history?
 - How does Deployment ensure zero downtime?
-

5. What is a Kubernetes Service and why is it needed?

Answer:

A Service exposes a group of Pods and provides **stable networking**.

Why needed:

- Pod IPs are dynamic
- Service provides fixed IP/DNS

Follow-up Questions:

- How does service discover Pods?
 - What is selector?
 - What happens if selector is wrong?
 - Difference between ClusterIP and Headless Service?
-

6. What are different types of Services?**Answer:**

- **ClusterIP (default):** Internal communication
 - **NodePort:** Exposes service on node IP + port
 - **LoadBalancer:** Uses cloud provider LB
 - **ExternalName:** Maps service to external DNS
-

Follow-up Questions:

- When to use NodePort vs LoadBalancer?
 - How does LoadBalancer work in AWS (ELB)?
 - What is headless service?
 - What is service mesh alternative?
-

7. What are ConfigMaps and Secrets?**Answer:**

- **ConfigMap:** Stores non-sensitive data (env variables, configs)
 - **Secret:** Stores sensitive data (passwords, API keys, TLS certs)
-

Follow-up Questions:

- Are secrets encrypted by default?
 - How do you secure secrets in production?
 - Difference between env and volume mounting?
 - What is external secret management (AWS Secrets Manager)?
-

8. What are Namespaces?

Answer:

Namespace is a logical separation within a cluster.

Used for:

- Multi-team environments
- Resource isolation

Example:

```
kubectl create namespace dev
```

Follow-up Questions:

- Can Pods across namespaces communicate?
 - What is default namespace?
 - How do you enforce limits per namespace?
 - What is ResourceQuota?
-

9. What are Labels and Selectors?

Answer:

- **Labels:** Key-value pairs assigned to objects
- **Selectors:** Used to filter/select objects

Example:

labels:

app: nginx

env: production

Follow-up Questions:

- Difference between equality and set-based selectors?
 - How do services use selectors?
 - What happens if labels change?
 - Can a Pod have multiple labels?
-

10. What are Persistent Volumes (PV) and Persistent Volume Claims (PVC)?

Answer:

- **PV:** Cluster-level storage resource
- **PVC:** Request for storage by Pods

Flow:

Pod → PVC → PV

- PV exists at **cluster level**
 - PVC is used by Pods to request storage
-

Follow-up Questions:

- What is StorageClass?
 - Static vs dynamic provisioning?
 - What is access mode (ReadWriteOnce, ReadOnlyMany)?
 - What happens if PVC is deleted?
-

Real Interview Insight (Important for You)

At your experience level (6–8 years), interviewer won't stop at definitions.

They will ask like:

👉 “You said Service gives stable IP — how exactly does kube-proxy implement that?”

Strong answer:

"kube-proxy uses iptables or IPVS to route traffic from Service IP to backend Pods dynamically."

1. What is a DaemonSet?

Answer:

A DaemonSet ensures that **a Pod runs on every node** in the cluster.

- If a new node is added → Pod is automatically deployed
- If a node is removed → Pod is removed

Use cases:

- Log collection (Fluentd)
 - Monitoring agents (Prometheus Node Exporter)
 - Security agents
-

Follow-up Questions:

- Difference between DaemonSet and Deployment?
 - Can you run DaemonSet on specific nodes only?
 - How do you update DaemonSet?
 - What happens during node scaling?
-

2. What is a Deployment?

Answer:

A Deployment manages stateless applications and ensures:

- Desired number of Pods
- Rolling updates
- Rollbacks
- Self-healing

Follow-up Questions:

- Deployment vs ReplicaSet?
 - Deployment vs StatefulSet?
 - What is rollout strategy?
 - What happens if Pods fail continuously?
-

3. What is a StatefulSet?**Answer:**

StatefulSet is used for **stateful applications** (like databases).

Features:

- Stable Pod names (pod-0, pod-1)
 - Persistent storage (via PVC)
 - Ordered deployment and scaling
 - Stable network identity
-

Follow-up Questions:

- Why not use Deployment for databases?
 - How does StatefulSet handle scaling?
 - What is headless service in StatefulSet?
 - Real-time example (MongoDB, MySQL)?
-

4. What is a Service Account?**Answer:**

A Service Account provides an **identity to Pods**.

- Used for authentication with Kubernetes API
- Works with RBAC for permissions

- In cloud (like AWS), mapped to IAM roles
-

Follow-up Questions:

- Difference between user and service account?
 - How does IRSA work in AWS EKS?
 - Where are service account tokens stored?
 - How do you secure service accounts?
-

5. What is a Pod?

Answer:

A Pod is the **smallest execution unit** in Kubernetes.

- Contains one or more containers
 - Shares:
 - Network
 - Storage
 - Ephemeral by nature
-

Follow-up Questions:

- What happens when Pod restarts?
 - Pod vs container?
 - What is Pod lifecycle?
 - What is multi-container Pod use case?
-

6. What are different types of Services?

Answer:

- **ClusterIP:** Internal communication

- **NodePort:** Accessible via node IP + port
 - **LoadBalancer:** Cloud-based external access
 - **ExternalName:** Maps to external DNS
-

Follow-up Questions:

- NodePort vs LoadBalancer in AWS?
 - What is headless service?
 - How does kube-proxy route traffic?
 - Can you expose without service?
-

7. What is a Sidecar Container?

Answer:

A sidecar container runs alongside the main container in the same Pod.

Use cases:

- Log collection
 - Proxy (Istio Envoy)
 - Monitoring
-

Follow-up Questions:

- Sidecar vs Init container?
 - Can sidecar restart independently?
 - Real-world use case (service mesh)?
 - Resource sharing behavior?
-

8. What is CrashLoopBackOff?

Answer:

CrashLoopBackOff means:

- Pod is failing repeatedly
- Kubernetes keeps restarting it

Common causes:

- Application crash
 - Wrong config
 - Insufficient resources
 - Dependency failure
-

Follow-up Questions:

- How do you debug CrashLoopBackOff?
 - Difference between ImagePullBackOff vs CrashLoopBackOff?
 - What logs to check first?
 - How to prevent it?
-

9. What is a ConfigMap?

Answer:

ConfigMap stores **non-sensitive configuration data**.

- Environment variables
 - Config files
-

Follow-up Questions:

- How do you mount ConfigMap?
 - Env vs volume usage?
 - What happens when ConfigMap changes?
 - Can Pods auto-reload config?
-

10. What is a Secret?

Answer:

Secret stores **sensitive data**:

- Passwords
 - API keys
 - Certificates
-

Follow-up Questions:

- Are Kubernetes Secrets encrypted?
 - How to secure Secrets in production?
 - Base64 encoding vs encryption?
 - Integration with AWS Secrets Manager?
-

Final Insight (Very Important for You)

Now you have **30 questions (Part 1 + 2 + 3)**.

But interviews won't be like:

✗ "What is Pod?"

They'll ask:

👉 "Your Pod is crashing in production, what will you do step by step?"

1. How can EC2 access an S3 bucket?

Answer:

The recommended way is using an **IAM Role**:

- Create an IAM Role with S3 permissions
 - Attach the role to the EC2 instance
 - Use AWS CLI/SDK inside EC2 to access S3
-

Follow-up Questions:

- Why not use access keys on EC2?
 - How does IAM Role provide temporary credentials?
 - What is Instance Metadata Service (IMDS)?
 - What is IMDSv2?
-

2. What is Trust Relationship vs IAM Policy?

Answer:

- **IAM Policy:** Defines *what actions are allowed* (e.g., S3 read/write)
- **Trust Relationship:** Defines *who can assume the role*

👉 In simple terms:

- Policy = Permissions
 - Trust = Who can use those permissions
-

Follow-up Questions:

- What is AssumeRole?
 - Can a role assume another role?
 - Cross-account role access how works?
 - What is sts:AssumeRole?
-

3. How does EC2 access S3 without internet?

Answer:

Use a **VPC Endpoint (Gateway Endpoint for S3)**:

- No Internet Gateway or NAT required
 - Private connection within AWS network
 - More secure and cost-effective
-

Follow-up Questions:

- Difference between Gateway vs Interface endpoint?
 - What is PrivateLink?
 - How to restrict access using endpoint policies?
 - Does it reduce data transfer cost?
-

4. How do you manage secrets in AWS?

Answer:

Use:

- **AWS Secrets Manager** (recommended)
- Alternative: SSM Parameter Store

Features:

- Secure storage
 - Encryption (KMS)
 - Automatic rotation (via Lambda)
-

Follow-up Questions:

- Secrets Manager vs Parameter Store?
 - How does rotation work?
 - How do applications fetch secrets securely?
 - Integration with Kubernetes (External Secrets)?
-

5. Archive data and delete after 1 year – how?

Answer:

Use **S3 Lifecycle Policy + Storage Class**:

- Store data in:

- **S3 Glacier / Glacier Deep Archive**
 - Configure lifecycle rule:
 - Transition → Glacier
 - Expire → after 365 days
-

Follow-up Questions:

- Glacier vs Deep Archive difference?
 - Retrieval time and cost?
 - Can lifecycle rules be version-specific?
 - What is Intelligent-Tiering?
-

6. How to share data between two S3 buckets?

Answer:

Options:

- **Same account:**
 - Bucket policy or IAM role
 - **Cross-account:**
 - Bucket policy + IAM role ARN
-

Follow-up Questions:

- How does cross-account access work?
 - What is bucket policy vs IAM policy?
 - Can you restrict by IP or VPC?
 - What is S3 replication?
-

7. Difference between ALB and NLB?

Answer:

Application Load Balancer (ALB):

- Layer 7 (HTTP/HTTPS)
- Path-based routing
- Used for web apps, microservices

Network Load Balancer (NLB):

- Layer 4 (TCP/UDP)
 - High performance, low latency
 - Used for real-time apps, gaming, IoT
-

Follow-up Questions:

- When to use ALB vs NLB?
 - What is target group?
 - Can ALB handle WebSockets?
 - What is sticky session?
-

8. What is Assume Role in IAM?

Answer:

Assume Role allows one entity to **temporarily gain permissions of another role.**

Example:

- User/Service A assumes Role B
 - Gets temporary credentials
 - Performs actions using Role B permissions
-

Follow-up Questions:

- What is STS (Security Token Service)?

- What is role chaining?
 - How long are temporary credentials valid?
 - Real use case (cross-account CI/CD)?
-

9. What is a Subnet?

Answer:

A Subnet is a **logical division of a VPC network**.

Types:

- **Public Subnet:** Has route to Internet Gateway
 - **Private Subnet:** No direct internet access
-

Follow-up Questions:

- Difference between subnet and VPC?
 - What is route table?
 - What is NAT Gateway?
 - How do you design multi-AZ architecture?
-

10. How do you pre-install software on EC2?

Answer:

Use **User Data script**:

- Runs during instance launch
- Used to install:
 - Docker
 - Jenkins
 - Packages, configs

Example:

```
#!/bin/bash
yum install docker -y
systemctl start docker
```

Follow-up Questions:

- Difference between User Data and AMI?
- Does user data run on reboot?
- How do you debug failed user data?
- What is Launch Template?

1. When should we use NACL vs Security Group?

Answer:

Feature	NACL	Security Group
---------	------	----------------

Level	Subnet	Instance
-------	--------	----------

Type	Stateless	Stateful
------	-----------	----------

Rules	Allow & Deny	Allow only
-------	--------------	------------

- Use **NACL** → coarse-grained control (subnet level)
 - Use **Security Group** → fine-grained control (instance level)
-

Follow-up Questions:

- What happens if NACL allows but SG denies?
 - Can you block specific IP using SG?
 - Order of evaluation in NACL?
 - Real-world example of NACL usage?
-

2. Default behavior of NACL and Security Group?

Answer:

- **NACL:**
 - Stateless
 - Default → allow all (custom NACL depends)
 - Rules evaluated in order
 - **Security Group:**
 - Stateful
 - Default → deny inbound, allow outbound
-

Follow-up Questions:

- Why is SG stateful important?
 - How does return traffic work in NACL?
 - What happens if outbound is blocked?
 - Troubleshooting connectivity steps?
-

3. What is a VPC?

Answer:

A VPC (Virtual Private Cloud) is an **isolated network environment** in AWS where you launch resources.

- You control:
 - IP range (CIDR)
 - Subnets
 - Routing
 - Security
-

Follow-up Questions:

- What is CIDR block?
- Can VPC span regions?

- Default VPC vs custom VPC?
 - How many VPCs per region?
-

4. Types of Subnets?

Answer:

- **Public Subnet:**
 - Has route to Internet Gateway
 - **Private Subnet:**
 - No direct internet access
-

Follow-up Questions:

- How does private subnet access internet?
 - What is NAT Gateway?
 - Can private subnet have public IP?
 - Multi-AZ subnet design?
-

5. How do you launch a public EC2 instance?

Answer:

Steps:

1. Create VPC
 2. Create Subnet
 3. Attach Internet Gateway
 4. Update Route Table (0.0.0.0/0 → IGW)
 5. Launch EC2 in public subnet
-

Follow-up Questions:

- What if no public IP assigned?
 - How to make existing instance public?
 - Security group requirements?
 - What is Elastic IP?
-

6. Can we change private IP of EC2?

Answer:

✗ No — primary private IP cannot be changed after launch.

✓ You can:

- Add secondary private IPs
 - Change public IP (on restart)
-

Follow-up Questions:

- What is Elastic IP?
 - Can we move IP between instances?
 - What happens on stop/start?
 - Use case of secondary IP?
-

7. How to connect EC2 to S3?

Answer:

Options:

- IAM Role (best practice)
 - Access keys (not recommended)
 - VPC Endpoint (for private EC2)
-

Follow-up Questions:

- Why avoid access keys?
 - How IAM role works internally?
 - What if EC2 is private?
 - Debug S3 access issue?
-

8. How to connect two VPCs?

Answer:

- **VPC Peering** → for few VPCs
 - **Transit Gateway** → for many VPCs (hub-spoke)
-

Follow-up Questions:

- Can VPC peering be transitive?
 - What is CIDR overlap issue?
 - Difference between TGW and peering?
 - Cross-region VPC connection?
-

9. VPC Peering vs Transit Gateway?

Answer:

- **VPC Peering:**
 - One-to-one connection
 - Complex at scale
 - **Transit Gateway:**
 - Hub-and-spoke model
 - Scalable and centralized routing
-

Follow-up Questions:

- Cost comparison?
 - Route table complexity?
 - TGW limits?
 - Real-world architecture?
-

10. Which service is used for structured data (row/column)?

Answer:

Use **Amazon RDS**.

- Relational database
 - Stores data in tables (rows & columns)
-

Follow-up Questions:

- RDS vs DynamoDB?
- What is Multi-AZ in RDS?
- Backup and restore?
- Scaling options?

1. What is User Data in EC2?

Answer:

User Data is a script that runs **automatically when an EC2 instance starts for the first time**.

- Used for:
 - Installing software
 - Configuring environment
 - Bootstrapping servers

Example:

```
#!/bin/bash
yum install docker -y
systemctl start docker
```

Follow-up Questions:

- Does user data run on every restart?
 - How do you debug failed user data?
 - Difference between User Data and AMI?
 - How to make user data run again?
-

2. What is Elastic IP (EIP)?**Answer:**

Elastic IP is a **static public IPv4 address** in AWS.

- Used when:
 - You need a fixed public IP
 - Unlike normal public IP:
 - Does NOT change on stop/start
-

Follow-up Questions:

- Is EIP free?
 - Can you attach EIP to multiple instances?
 - What happens if EIP is not attached?
 - EIP vs Public IP?
-

3. How do you assign a static IP to EC2?**Answer:**

By using **Elastic IP**:

1. Allocate EIP
2. Associate it with EC2 instance

Follow-up Questions:

- Can private IP be static?
 - Can EIP move across regions?
 - What is secondary private IP?
 - Use case of static IP?
-

4. What are different types of EC2 instances?**Answer:**

Based on use case:

- **General Purpose (T, M)** → Balanced
 - **Compute Optimized (C)** → CPU intensive
 - **Memory Optimized (R)** → RAM heavy workloads
 - **Storage Optimized (I, D)** → High disk I/O
 - **Accelerated (P, G)** → GPU workloads
-

Follow-up Questions:

- How do you choose instance type?
 - What is burstable instance (T family)?
 - What is Spot vs On-Demand?
 - Cost optimization strategies?
-

5. What is default route in main route table?**Answer:**

Every VPC has a **main route table** with default route:

- **Local route** → allows communication within VPC

Example:

Destination: VPC CIDR → Target: local

Follow-up Questions:

- Can you delete main route table?
 - What is route table association?
 - What happens without route table?
 - How routing works in AWS?
-

6. What is a VPC Endpoint?

Answer:

VPC Endpoint allows **private communication between VPC and AWS services** without internet.

Example:

- EC2 → S3 (without NAT/IGW)

Benefits:

- More secure
 - Cost efficient
 - No internet exposure
-

Follow-up Questions:

- Types of endpoints?
 - Gateway vs Interface endpoint?
 - What is PrivateLink?
 - Endpoint policy usage?
-

7. How to allow EC2 outbound internet but block inbound?

Answer:

Use **NAT Gateway** in private subnet:

- EC2 → Internet ✓
 - Internet → EC2 ✗
-

Follow-up Questions:

- NAT Gateway vs NAT Instance?
 - Where should NAT be placed?
 - Cost considerations?
 - How to debug outbound failure?
-

8. What is a Target Group?

Answer:

Target Group is a **collection of EC2 instances (or targets)** behind a Load Balancer.

- Load Balancer routes traffic → Target Group → EC2
-

Follow-up Questions:

- What is health check?
 - Can target group have different ports?
 - ALB vs NLB target groups?
 - What happens if instance unhealthy?
-

9. What is Auto Scaling Group (ASG)?

Answer:

ASG automatically manages EC2 instances:

- Maintains desired capacity

- Replaces unhealthy instances
 - Scales based on demand
-

Follow-up Questions:

- What is desired, min, max capacity?
 - How does ASG detect unhealthy instance?
 - Integration with Load Balancer?
 - What is cooldown period?
-

10. Types of Auto Scaling Policies?

Answer:

- **Simple Scaling:**
 - Fixed scaling + cooldown
 - **Step Scaling:**
 - Scale based on threshold levels
 - **Target Tracking:**
 - Maintains target metric (e.g., CPU = 50%)
-

Follow-up Questions:

- Which policy is most used in production?
- Difference between step vs target tracking?
- What metrics can be used?
- What happens during sudden spike?

1. When should you use NACL vs Security Group?

Answer:

- **NACL (Network ACL):**

- Works at **subnet level**
- Used for **coarse-grained control**
- Useful when you want to allow/deny traffic at network boundary
- **Security Group:**
 - Works at **instance level**
 - Used for **fine-grained control**
 - Acts like a virtual firewall for EC2

Traffic Flow:

Internet → NACL → Subnet → Security Group → EC2

Follow-up Questions:

- What happens if NACL allows but Security Group denies?
 - Can Security Group block a specific IP?
 - Can NACL be attached to multiple subnets?
 - Real production use case for NACL?
-

2. What are default properties of NACL and Security Group?

Answer:

NACL:

- Stateless (no memory of requests)
- Requires **both inbound & outbound rules**
- Supports **Allow and Deny**
- Rules evaluated **in order (low to high)**

Security Group:

- Stateful (remembers traffic)
- Only **inbound rules needed for return traffic**

- Supports **Allow only**
- Evaluates rules together

Default:

- SG → Deny inbound, allow outbound
-

Follow-up Questions:

- Why is stateful behavior important?
 - How does return traffic work in NACL?
 - Which is more secure in production?
 - Troubleshooting connectivity issue steps?
-

3. What is a VPC?

Answer:

A VPC (Virtual Private Cloud) is a **logically isolated network** in AWS.

- You control:
 - IP range (CIDR)
 - Subnets
 - Routing
 - Security

👉 Think of it as your **private data center inside AWS**

Follow-up Questions:

- What is CIDR block?
- Can VPC span multiple regions?
- Default VPC vs custom VPC?
- What is VPC peering?

4. How many types of subnets are there?

Answer:

- **Public Subnet:**
 - Route table has **Internet Gateway (IGW)**
 - Direct internet access
 - **Private Subnet:**
 - No direct IGW route
 - Uses **NAT Gateway** for outbound internet
-

Follow-up Questions:

- Can private subnet access internet?
 - What is NAT Gateway vs NAT Instance?
 - Can public subnet exist without IGW?
 - Multi-AZ subnet design?
-

5. How do you launch a public EC2 instance?

Answer:

Steps:

1. Create VPC
2. Create Subnet
3. Create & attach **Internet Gateway**
4. Add route:

0.0.0.0/0 → IGW

5. Associate route table with subnet
6. Launch EC2 in that subnet

Follow-up Questions:

- What if public IP is not assigned?
 - How to convert private EC2 to public?
 - Security group configuration needed?
 - What is Elastic IP?
-

6. Can we change private IP of EC2?**Answer:**

✗ Primary private IP → **Cannot be changed**

✓ You can:

- Add secondary private IPs
 - Change public IP (on restart)
-

Follow-up Questions:

- What is Elastic IP?
 - Can we move IP across instances?
 - What happens during stop/start?
 - Use case of secondary private IP?
-

7. How to connect EC2 to S3?**Answer:****Recommended:**

- **IAM Role attached to EC2**

Other methods:

- Access keys via aws configure (not recommended)

- **VPC Endpoint** (for private EC2)
-

Follow-up Questions:

- Why avoid access keys?
 - How IAM role works internally?
 - What is IMDS?
 - How to debug S3 access failure?
-

8. How to connect two VPCs?

Answer:

- **VPC Peering:**
 - Direct connection (1-to-1)
 - **Transit Gateway:**
 - Hub-and-spoke model
 - Scales for multiple VPCs
-

Follow-up Questions:

- Is VPC peering transitive?
 - What happens if CIDR overlaps?
 - Cross-region connectivity options?
 - Cost comparison?
-

9. When to use VPC Peering vs Transit Gateway?

Answer:

- Use **VPC Peering:**
 - Few VPCs

- Simple architecture
 - Use **Transit Gateway**:
 - Many VPCs
 - Centralized routing
 - Scalable architecture
-

Follow-up Questions:

- Real production use case?
 - How routing works in TGW?
 - Limits of VPC peering?
 - TGW pricing model?
-

10. Which AWS service is used for structured (row-column) data?

Answer:

Use **Amazon RDS**

- Relational database
 - Stores data in tables (rows & columns)
-

Follow-up Questions:

- RDS vs DynamoDB?
 - What is Multi-AZ?
 - Backup and restore strategy?
 - Read replicas vs Multi-AZ?
-

What Interviewers Actually Expect (Important)

Right now your answers are **correct**, but to crack interviews:

Instead of:

“Use IAM role to access S3”

Say:

“I’ll attach IAM role, verify trust policy, check instance metadata credentials, then test with AWS CLI and debug using CloudTrail if access fails.”

👉 That level = **selected candidate**

🚀 Next Step (Strong Recommendation)

You’ve now completed:

- ☒ AWS (Part 1, 2, 3)
- ☒ Kubernetes (Part 1, 2, 3)

Now the **missing piece**:

👉 Say: “**scenario questions**”

I’ll give you:

- 🧠 Real DevOps production issues
- 🧠 Step-by-step debugging answers
- 🧠 Exactly what comes in TCS, Tech Mahindra, product companies

This is what you need to clear **final rounds**.

This chat is nearing its li

1. Write a Jenkins Pipeline (Jenkinsfile)

Answer:

A Jenkinsfile defines the CI/CD pipeline using **Declarative or Scripted syntax**.

Basic structure:

```
pipeline {  
  agent any  
  
  stages {
```

```
stage('Build') {
    steps {
        echo 'Building...'
    }
}

stage('Test') {
    steps {
        echo 'Testing...'
    }
}

stage('Deploy') {
    steps {
        echo 'Deploying...'
    }
}

post {
    always {
        echo 'Pipeline completed'
    }
}
}
```

Follow-up Questions:

- Declarative vs Scripted pipeline?
- What is agent in Jenkins?
- What is post block used for?
- How do you handle credentials in Jenkins?

2. What is a Multibranch Pipeline?

Answer:

A Multibranch Pipeline automatically:

- Detects branches in SCM (GitHub/GitLab)
- Creates pipelines for each branch
- Runs builds independently

Use case:

- Different developers working on different branches
 - Each branch gets its own CI/CD pipeline
-

Follow-up Questions:

- How does Jenkins detect new branches?
 - What is branch indexing?
 - Difference between multibranch and normal pipeline?
 - How do you deploy different environments from branches?
-

3. Write Jenkinsfile to print "Hello World"

Answer:

```
pipeline {
  agent any
  stages {
    stage('Hello') {
      steps {
        echo 'Hello World'
      }
    }
  }
}
```

Follow-up Questions:

- Difference between echo and sh?

- Can you run shell commands here?
 - How to run Python/Java code in pipeline?
-

4. What is CI/CD? Difference between CI, CD (Delivery vs Deployment)?

Answer:

- **CI (Continuous Integration):**
 - Developers frequently merge code
 - Automated build & testing
 - **CD (Continuous Delivery):**
 - Code is always ready for deployment
 - Requires manual approval
 - **CD (Continuous Deployment):**
 - Fully automated deployment
 - No manual approval
-

Follow-up Questions:

- Which one is used in production?
 - What are risks of continuous deployment?
 - How do you implement approvals?
 - Tools used in CI/CD pipeline?
-

5. What is Jenkins Shared Library?

Answer:

Shared Library allows reuse of common pipeline code.

- Write once → reuse across pipelines
- Stored in Git repo

- Helps standardization
-

Follow-up Questions:

- What are vars and src folders?
 - How do you load shared library?
 - Benefits in large organizations?
 - Versioning of shared libraries?
-

6. What is Webhook in Jenkins?

Answer:

Webhook triggers Jenkins automatically when code changes happen in SCM.

Flow:

Git push → Webhook → Jenkins → Pipeline triggered

Follow-up Questions:

- Poll SCM vs Webhook?
 - How to configure webhook in GitHub?
 - What happens if webhook fails?
 - Security considerations?
-

7. How do you integrate Jenkins with Slack (or notifications)?

Answer:

- Use Jenkins plugins (Slack plugin, Email plugin)
- Configure webhook/token
- Send notifications on:
 - Success

- Failure
 - Build status
-

Follow-up Questions:

- What events do you notify?
 - Slack vs Email – which is better?
 - How to customize messages?
 - Can you send logs in notifications?
-

8. Explain Multibranch Pipeline again (Practical View)

Answer:

- Developers create feature branches
 - Jenkins auto-detects branches
 - Runs pipeline per branch
 - After testing → merge to main → deploy
-

Follow-up Questions:

- How do you handle PR builds?
 - Branch strategy (GitFlow)?
 - Can you restrict pipeline execution?
 - Cleanup old branches?
-

9. What is Parameterized Pipeline?

Answer:

Allows user input before pipeline runs.

Example:

- Environment (dev/test/prod)
 - Version number
 - Custom values
-

Follow-up Questions:

- Types of parameters?
 - How to use parameters in script?
 - Can parameters be dynamic?
 - Use case in real pipeline?
-

10. Declarative vs Scripted Pipeline?

Answer:

- **Declarative:**
 - Structured, easier
 - Recommended
 - **Scripted:**
 - Flexible (Groovy-based)
 - Complex logic possible
-

Follow-up Questions:

- Which one do you use in projects?
 - When to use scripted?
 - Can we mix both?
 - Limitations of declarative?
-

11. How to continue pipeline even if a stage fails?

Answer:

Use `catchError` or `try-catch`:

```
stage('Test') {  
  steps {  
    catchError(buildResult: 'SUCCESS', stageResult: 'FAILURE') {  
      sh 'exit 1'  
    }  
  }  
}
```

- Pipeline continues even if stage fails
-

Follow-up Questions:

- Difference between unstable and failure?
 - When should pipeline stop?
 - How to handle partial failures?
 - Real-world use case?
-

 Real Interview Insight (Important for You)

At your level (6–8 yrs), interviewer will ask like:

👉 “Your Jenkins pipeline is failing at deploy stage — how will you debug?”

👉 “Webhook not triggering pipeline — what will you check?”

👉 “Pipeline is slow — how will you optimize?”

 Next Step (Highly Recommend)**Linux Interview Questions (Refined + Follow-ups)**

1. How do you check CPU and memory utilization?

Answer:

Common commands:

- top → Real-time CPU & memory usage
 - htop → Improved version of top
 - free -h → Memory usage
 - vmstat → System performance
 - mpstat → CPU stats
-

Follow-up Questions:

- Difference between top and htop?
 - What is load average?
 - How to identify high CPU process?
 - What will you check if server is slow?
-

2. Write a bash script to add two numbers

Answer:

```
#!/bin/bash
```

```
add() {  
    sum=$(( $1 + $2 ))  
    echo "Sum is: $sum"  
}
```

```
add 10 20
```

Follow-up Questions:

- How do you take user input?
- Difference between \$1 and \$@?
- How do you handle invalid input?

- How to return values from function?
-

3. Write a script to reverse a string

Answer:

```
#!/bin/bash
```

```
str="hello"
```

```
rev_str=$(echo $str | rev)
```

```
echo "Reversed: $rev_str"
```

Follow-up Questions:

- Can you do it without rev?
 - How to reverse a file?
 - What is awk usage here?
 - Performance considerations?
-

4. How do you rotate log files?

Answer:

Use **logrotate**:

- Automatically rotates logs
- Compresses old logs
- Deletes after retention period

Example config:

```
/var/log/app.log {  
    daily  
    rotate 7  
    compress
```

```
missingok  
}
```

Follow-up Questions:

- Where is logrotate config stored?
 - How to force log rotation?
 - What is retention policy?
 - Real-world use case?
-

5. Command to list all processes

Answer:

- `ps -ef`
 - `ps aux`
-

Follow-up Questions:

- Difference between `ps` and `top`?
 - How to kill a process?
 - What is zombie process?
 - How to find parent process?
-

6. What is \$ in Linux?

Answer:

- Used to access variable value

Example:

```
name="DevOps"  
echo $name
```

Follow-up Questions:

- Difference between \$, \$\$, \$??
 - What is \$PATH?
 - Environment vs local variables?
 - How to export variable?
-

7. How to check CPU, memory, and system load?

Answer:

- top
 - uptime → shows load average
 - free -h
 - sar (if installed)
-

Load Average Concept (Important)

$$\text{Load Average} = \frac{\text{Number of running processes}}{\text{Number of CPU cores}}$$

 If load > CPU cores → system is overloaded

Follow-up Questions:

- What does 1, 5, 15 mean in load average?
 - Acceptable load for system?
 - How to reduce load?
 - Debug high load issue?
-

8. How to check mounted volumes and disk space?

Answer:

- `df -h` → disk usage
 - `lsblk` → block devices
 - `mount` → mounted file systems
-

Follow-up Questions:

- Difference between `df` and `du`?
 - How to find large files?
 - What happens if disk is full?
 - Cleanup strategies?
-

9. Find and replace a word in a file

Answer:

```
sed -i 's/oldword/newword/g' file.txt
```

Follow-up Questions:

- What does `-i` mean?
 - Difference between `sed` and `awk`?
 - Replace in multiple files?
 - Backup before replace?
-

10. How to check hostname?

Answer:

```
hostname
```

Follow-up Questions:

- How to change hostname?

- Where is hostname stored?
 - Difference between hostname and IP?
 - DNS resolution flow?
-

11. How to check OS version?

Answer:

- `uname -a`
 - `cat /etc/os-release`
-

Follow-up Questions:

- Difference between kernel and OS?
 - How to check architecture?
 - What is kernel version?
 - Upgrade process?
-

Additional Must-Know Commands

- `chmod` → change permissions
 - `chown` → change ownership
 - `cat`, `less`, `tail` → file reading
 - `vi` / `nano` → editing
 - `grep` → search
-

Real Interview Twist (Important)

They won't ask:

✗ "What is `df` command?"

They will ask:

👉 “Disk is 100% full — what will you do?”

👉 “CPU is 100% — how will you debug?”

👉 “Process stuck — how will you handle?”

🚀 **Next Step (Most Important for**

💎 **Terraform Interview Questions (Refined + Follow-ups)**

1. Where do you store the Terraform state file?

Answer:

- **Local backend** → for testing
- **Remote backend (recommended):**
 - AWS S3 (most common)
 - Azure Blob
 - GCS bucket

👉 In production:

- Store in **S3**
- Use **DynamoDB for state locking**

Follow-up Questions:

- Why remote backend is required?
- What is state locking?
- What happens without locking?
- How to secure state file?

2. What happens if infrastructure is deleted manually outside Terraform?

Answer:

Terraform compares:

👉 **Config (.tf files) vs State file**

- If resource exists in state but not in cloud:
 - Terraform **recreates it on next apply**

👉 Terraform does NOT automatically detect manual changes unless refreshed

Follow-up Questions:

- What is terraform refresh?
 - What is drift?
 - How to detect drift in production?
 - How to avoid manual changes?
-

3. What is terraform import?

Answer:

Used to bring **existing resources into Terraform state**.

Example:

```
terraform import aws_instance.myec2 i-1234567890
```

👉 After import:

- Terraform starts managing that resource
-

Follow-up Questions:

- Does import create config automatically?
 - What is terraform state command?
 - What is state mv?
 - Real-world use case?
-

4. Write Terraform code for EC2

Answer:

```
resource "aws_instance" "example" {  
  ami      = "ami-123456"  
  instance_type = "t2.micro"  
  
  tags = {  
    Name = "MyEC2"  
  }  
}
```

Follow-up Questions:

- How do you attach volume?
 - How to pass variables?
 - How to use count or for_each?
 - How to provision using user_data?
-

5. What Terraform version have you used?

Answer:

- Mention your actual experience
 - Example:
 - “I’ve worked with Terraform 1.x versions with remote backend and modules”
-

Follow-up Questions:

- What changed from 0.11 to 0.12+?
- How do you upgrade versions?
- What is provider versioning?
- What is required_version?

6. How do you manage state files in AWS?

Answer:

Best practice:

- Store in **S3 bucket**
- Enable:
 - Versioning
 - Encryption
- Use **DynamoDB** for locking

For environments:

- Use separate paths:
 - dev/
 - test/
 - prod/

Follow-up Questions:

- Workspace vs folder structure?
- How to isolate environments?
- What is backend config?
- How to migrate state?

7. What challenges have you faced in Terraform?

Answer (example):

- State conflicts due to no locking
- Drift due to manual changes
- Complex modules with `for_each`

- Multi-account / multi-region setup

👉 Example answer:

“We faced state corruption due to parallel execution. We fixed it using DynamoDB locking and proper pipeline control.”

Follow-up Questions:

- How did you debug Terraform failure?
 - What is plan vs apply issue?
 - Handling dependency issues?
 - CI/CD integration challenges?
-

8. What are Terraform modules?

Answer:

Modules = reusable code blocks

- Write once → reuse multiple times
- Helps standardization

Example:

- VPC module used across environments
-

Follow-up Questions:

- Local vs remote modules?
 - How to pass variables?
 - How to version modules?
 - Real-world usage?
-

9. How do you recover corrupted state file?

Answer:

- Use **S3 versioning** → restore previous version
 - Backup strategy
 - Remote backend recovery
-

Follow-up Questions:

- What if state is deleted?
 - How to manually fix state?
 - What is terraform state rm?
 - Disaster recovery strategy?
-

10. What is terraform taint?

Answer:

Marks resource for **recreation** on next apply

terraform taint aws_instance.example

👉 Used when resource is partially broken

Follow-up Questions:

- Difference between taint and destroy?
 - What is -replace option?
 - When to use taint in real scenario?
 - Risks of taint?
-

11. How to destroy a specific resource?

Answer:

terraform destroy -target=aws_instance.example

👉 Deletes only specific resource

Follow-up Questions:

- Is -target recommended in production?
 - What are risks?
 - Alternative approach?
 - Dependency issues?
-

🔗 Real Interview Insight (Important for You)

They won't ask:

✗ "What is module?"

They will ask:

👉 "State file got corrupted during deployment — what will you do?"

👉 "Two engineers ran terraform apply simultaneously — what happens?"

👉 "Infra drift detected — how to fix?"

🎯 Docker Interview Questions (Refined + Follow-ups)

1. Difference between ARG and ENV

Answer:

Feature	ARG	ENV
Scope	Build-time only	Runtime + build-time
Availability	Only during image build	Available in container
Use case	Versioning, build configs	App configs, environment values

👉 Example:

ARG VERSION=1.0

ENV APP_ENV=prod

Follow-up Questions:

- Can ARG be accessed in running container?
 - Can ENV override ARG?
 - How do you pass ENV at runtime?
 - Security concerns with ENV?
-

2. Difference between RUN and CMD

Answer:

- **RUN**
 - Executes during **image build**
 - Creates new image layers
 - Used for installing packages
- **CMD**
 - Executes during **container start**
 - Defines default command

📄 Example:

RUN apt-get install -y nginx

CMD ["nginx", "-g", "daemon off;"]

Follow-up Questions:

- Difference between CMD and ENTRYPOINT?
 - Can CMD be overridden?
 - What happens if multiple CMDs exist?
-

3. What is Multi-stage Docker build?

Answer:

Used to:

- 👉 Reduce image size
- 👉 Separate build & runtime environments

Example:

FROM node:18 AS builder

WORKDIR /app

COPY . .

RUN npm install && npm run build

FROM nginx:alpine

COPY --from=builder /app/build /usr/share/nginx/html

Follow-up Questions:

- Why is image size important?
- How does it improve security?
- Real-world use case?

4. Dockerfile Best Practices

Answer:

- Use **lightweight base images** (alpine)
- Minimize layers (combine RUN commands)
- Avoid root user
- Use .dockerignore
- Use multi-stage builds
- Store secrets outside image

Follow-up Questions:

- How to reduce image size?
 - How to scan vulnerabilities?
 - How to manage secrets?
-

5. Difference between CMD and ENTRYPOINT

Answer:

CMD	ENTRYPOINT
Default command	Fixed command
Can be overridden easily	Harder to override
Used for arguments	Used for main execution

📄 Example:

ENTRYPOINT ["python"]

CMD ["app.py"]

Follow-up Questions:

- How to override ENTRYPOINT?
 - When to use both together?
 - Debugging ENTRYPOINT issues?
-

6. Difference between ADD and COPY

Answer:

COPY	ADD
Simple file copy	Advanced (URL, unzip)
Preferred	Less preferred

👉 Best practice:

👉 Always use **COPY** unless ADD features required

Follow-up Questions:

- Why avoid ADD?
 - Security implications?
 - When ADD is actually needed?
-

7. Why Docker containers are lightweight?

Answer:

- Share **host OS kernel**
- No full OS per container
- Only required dependencies included

👉 Result:

- Faster startup
 - Lower memory usage
-

Follow-up Questions:

- Container vs VM difference?
 - Kernel sharing risks?
 - Isolation level?
-

8. Docker Networking Types

Answer:

- **Bridge (default)**
- **Host**

- **Overlay** (for swarm / multi-host)
 - **Macvlan**
-

Follow-up Questions:

- When to use host network?
 - Difference between bridge & overlay?
 - How containers communicate?
-

9. Host Network Concept

Answer:

👉 In host network:

- Container shares **host IP**
 - No network isolation
-

Follow-up Questions:

- Pros and cons?
 - Security risks?
 - Use cases?
-

10. Docker Volume Mount Command

Answer:

`docker run -v /host/log:/var/log:rw nginx`

- `/host/log` → host path
 - `/var/log` → container path
 - `rw` → read-write
-

Follow-up Questions:

- Difference between bind mount and volume?
 - Where Docker volumes stored?
 - Data persistence strategy?
-

11. What is Docker Networking (Concept)

Answer:

Docker networking allows:

- Container ↔ Container communication
 - Container ↔ Host communication
 - External access
-

Follow-up Questions:

- How service discovery works?
 - DNS in Docker?
 - How Kubernetes networking differs?
-

Real Interview Twist (Important)

At your experience level, interviewer may ask:

- 👉 “Your container is running but app is not accessible — what will you check?”
 - 👉 “Image size is 1.5GB — how will you reduce it?”
 - 👉 “Container restarts continuously — debugging steps?”
 - 👉 “Port is exposed but not accessible — why?”
-

What you should focus now

You already covered:

- AWS
- Terraform
- Kubernetes
- CI/CD
- Linux
- Docker

👉 This is **full DevOps stack**

⚠ But interviews at your level test:

👉 **Troubleshooting + Architecture + Decision making**

🔥 **Next step**

Say:

👉 **“give me docker real-time scenarios”**

I’ll give you:

- 🔥 Production-level Docker issues
- 🔥 Debugging flow (step-by-step)
- 🔥 Exact answers to impress interviewer

💎 **DevOps Interview Questions (Advanced Mixed Set)**

1. What is the default Docker network?

Answer:

👉 Default network = **bridge**

- Used when no network is specified
- Containers communicate using internal IPs

Follow-up Questions:

- Difference between bridge and host?
 - How to inspect Docker network?
 - Can containers communicate across networks?
-

2. Can you delete an image if a container is running?

Answer:

✗ Normally: **No**

- Docker prevents deletion of image in use

✓ Force delete:

```
docker rmi -f <image_id>
```

⚠ Not recommended in production

Follow-up Questions:

- What happens to running container after force delete?
 - How to safely remove unused images?
 - Difference between prune and rmi?
-

3. Difference between Docker and Kubernetes

Answer:

Docker	Kubernetes
Containerization tool	Orchestration tool
Builds & runs containers	Manages containers at scale
Single host focus	Multi-node cluster

👉 Flow:

- Docker → build image
 - Kubernetes → deploy & scale
-

Follow-up Questions:

- Can Kubernetes run without Docker?
 - What is container runtime?
 - Difference between Docker Swarm & Kubernetes?
-

4. Container vs Virtual Machine

Answer:

Container	VM
Lightweight	Heavy
Shares host kernel	Full OS
Fast startup	Slow startup

Follow-up Questions:

- What is namespace & cgroups?
 - Security concerns in containers?
 - When to use VM over container?
-

5. How to run container as non-root user?

Answer:

In Dockerfile:

```
RUN groupadd appgroup && useradd -g appgroup appuser  
USER appuser
```

👉 Improves security

Follow-up Questions:

- Why avoid root user?
 - What is rootless Docker?
 - Permission issues troubleshooting?
-

🎯 AWS Questions

6. CloudWatch Alarms vs Events (EventBridge)

Answer:

- **CloudWatch Alarms**
 - Trigger based on metrics (CPU, memory)
 - **EventBridge (Events)**
 - Trigger based on events (API calls, state changes)
-

Follow-up Questions:

- Can alarm trigger Lambda?
 - Difference between logs and metrics?
 - Real-time monitoring setup?
-

7. What happens if VPC CIDRs overlap?

Answer:

✗ Communication fails (peering not possible)

👉 Solutions:

- Use **non-overlapping CIDR**

- Redesign network
 - Use **NAT / proxy workaround (rare cases)**
-

Follow-up Questions:

- CIDR planning strategy?
 - VPC peering limitations?
 - Transit Gateway vs Peering?
-

8. EC2 vs S3

Answer:

EC2 **S3**

Compute Storage

Virtual server Object storage

Follow-up Questions:

- When to use EBS vs S3?
 - S3 consistency model?
 - Storage classes use case?
-

9. Can EC2 be attached to multiple target groups?

Answer:

☒ Yes

- Same instance can serve multiple applications
 - Attached to multiple load balancers
-

Follow-up Questions:

- Use case?
 - Health check behavior?
 - Target group vs listener?
-

10. EC2 Instance Types**Answer:**

- General purpose (t, m)
 - Compute optimized (c)
 - Memory optimized (r)
 - Storage optimized (i, d)
 - GPU (p, g)
-

Follow-up Questions:

- How to choose instance type?
 - Spot vs on-demand vs reserved?
 - Cost optimization strategy?
-

11. S3 Storage Classes**Answer:**

- Standard
- Intelligent-Tiering
- Standard-IA
- One Zone-IA
- Glacier
- Glacier Deep Archive

Follow-up Questions:

- Lifecycle policies?
- Cost optimization?
- Retrieval time differences?

◆ Jenkins / CI-CD Questions

12. Declarative vs Scripted Pipeline

Answer:

Declarative Scripted

Simple Flexible

Structured Custom logic

👉 Mostly used: **Declarative**

Follow-up Questions:

- When to use scripted?
 - Pipeline syntax difference?
 - Error handling?
-

13. How to continue pipeline even if stage fails?

Answer:

Use:

```
catchError {  
  stage('Test') {  
    sh 'exit 1'
```

```
}  
}
```

Follow-up Questions:

- Difference between try-catch vs catchError?
 - Mark build unstable vs failed?
 - Post block usage?
-

14. How to send notifications in Jenkins?

Answer:

- Email (SMTP)
- Slack
- Webhooks

Example:

```
post {  
  failure {  
    mail to: 'team@example.com'  
  }  
}
```

Follow-up Questions:

- Slack integration steps?
 - Notification on success/failure?
 - Alert fatigue handling?
-

15. Continuous Delivery vs Continuous Deployment

Answer:

Delivery

Deployment

Manual approval Fully automated

Push to staging Push to production

Follow-up Questions:

- Which is safer?
 - Where approvals required?
 - Real-world pipeline design?
-

16. What is Jenkins Blue Ocean?

Answer:

- Modern UI for Jenkins
 - Visual pipeline representation
-

Follow-up Questions:

- Benefits over classic UI?
 - Is it production ready?
 - Alternatives?
-

17. Can we add new EC2 as Jenkins agent?

Answer:

☒ Yes

Steps:

- Install agent
- Configure SSH

- Add node in Jenkins
-

Follow-up Questions:

- Static vs dynamic agents?
 - Auto scaling agents?
 - Security setup?
-

18. How to migrate Jenkins to new server?

Answer:

- Backup:
 - /var/lib/jenkins
 - config.xml
 - plugins
 - Restore on new server
-

Follow-up Questions:

- Zero downtime migration?
 - Plugin compatibility issues?
 - Backup strategy?
-

Real Interview Focus (Critical for You)

At your level, questions will shift like:

- 👉 “Pipeline failed at stage 3 — how will you debug?”
- 👉 “VPC peering not working — what could be wrong?”
- 👉 “Container deployed but not reachable — why?”
- 👉 “Jenkins migration failed — rollback plan?”

◆ Linux Basic Commands (Interview Ready)

1. How do you list files in Linux?

Answer:

- `ls` → basic listing
 - `ls -l` → detailed view (permissions, owner, size)
 - `ls -lh` → human-readable sizes
 - `ls -a` → includes hidden files
-

Follow-up Questions:

- How to sort files by size?
 - How to list only directories?
 - Difference between `ls` and `find`?
-

2. How do you change directory?

Answer:

`cd /path/to/directory`

- `cd ..` → go back
 - `cd ~` → home directory
-

Follow-up Questions:

- Difference between absolute and relative path?
 - What does `cd -` do?
-

3. How to change file permissions?

Answer:

`chmod 755 file.txt`

- 7 → read + write + execute
 - 5 → read + execute
-

Follow-up Questions:

- What is symbolic mode (`chmod u+x`)?
 - What are special permissions (SUID, SGID)?
 - How to apply recursively?
-

4. How to change ownership of a file?

Answer:

`chown user:group file.txt`

Follow-up Questions:

- Difference between user and group ownership?
 - Recursive ownership change?
-

5. How to delete files?

Answer:

`rm file.txt`

`rm -r folder/`

`rm -rf folder/`

⚠ -rf is dangerous (force + recursive)

Follow-up Questions:

- How to delete safely?

- What is rm -i?
 - Recover deleted files?
-

6. How to check current directory?

Answer:

pwd

Follow-up Questions:

- Difference between \$PWD and pwd?
 - How shell maintains current directory?
-

7. How to create directory?

Answer:

mkdir new_folder
mkdir -p /path/to/folder

Follow-up Questions:

- What does -p do?
 - Create multiple directories?
-

8. How to move or rename files?

Answer:

mv old.txt new.txt
mv file.txt /new/location/

Follow-up Questions:

- Difference between mv and cp?

- What happens if file exists?
-

9. How to copy files?

Answer:

```
cp file1.txt file2.txt  
cp -r folder1 folder2
```

Follow-up Questions:

- Preserve permissions?
 - Copy only updated files?
-

10. How to view file content?

Answer:

```
cat file.txt  
less file.txt  
head file.txt  
tail file.txt
```

Follow-up Questions:

- Difference between cat and less?
 - How to monitor logs in real-time? (tail -f)
-

11. How to edit files?

Answer:

```
vi file.txt
```

- i → insert mode
- Esc + :wq → save & exit

- Esc + :q! → exit without saving
-

Follow-up Questions:

- Difference between vi and nano?
 - Search inside file in vi?
 - Replace text in vi?
-

Important Upgrade (For Your Experience Level)

These basics are expected, but interviews **won't stop here**.

They will move to:

- 👉 “CPU is high — how will you debug?”
 - 👉 “Disk is full — what steps?”
 - 👉 “Process is stuck — how to handle?”
 - 👉 “Application logs not visible — why?”
-

What you should prepare next

You already know:

- Basic Linux commands 

Now you must master:

-  Process troubleshooting (top, ps, kill)
-  Disk issues (df, du)
-  Logs (journalctl, tail -f)
-  Networking (netstat, ss, curl)

AWS + DevOps Interview Questions (Clean Answers)

1. VPN Setup in AWS

Answer:

Yes, I have worked on AWS VPN setup.

- Created:
 - Virtual Private Gateway (AWS side)
 - Customer Gateway (on-prem side)
- Configured:
 - Site-to-Site VPN connection
 - Route tables for private communication
- Ensured:
 - Secure encrypted tunnel between AWS VPC and on-prem data center

👉 Used when hybrid architecture is needed.

2. VPC Peering vs Communication

VPC Peering:

- Direct connection between 2 VPCs
- Low latency
- No transitive routing

Steps:

- Create peering connection
- Update route tables
- Allow traffic via security groups

Limitation:

- Not scalable for many VPCs

👉 Use **Transit Gateway** when multiple VPCs are involved.

3. What is VPC Endpoint?

Answer:

VPC Endpoint allows **private communication** between VPC and AWS services **without Internet/NAT**.

Types:

- Gateway Endpoint → S3, DynamoDB
- Interface Endpoint → PrivateLink (ENI-based)

📁 Example:

EC2 → S3 (without internet)

✓ More secure

✓ Cost-efficient (no NAT cost)

4. AWS Migration Tools Used

Answer:

- **DMS (Database Migration Service)** → DB migration
- **AWS Application Migration Service (MGN)** → lift & shift
- **CloudFormation / Terraform** → infra recreation
- **CodeBuild / CodePipeline** → deployment

📁 Strategy:

- Rehost (lift & shift)
- Replatform
- Re-architect

5. CloudWatch Alarm vs Event

Feature	CloudWatch Alarm	CloudWatch Events (EventBridge)
---------	------------------	---------------------------------

Purpose	Monitoring thresholds	Event-driven automation
---------	-----------------------	-------------------------

Example	CPU > 70%	Trigger Lambda
---------	-----------	----------------

Feature	CloudWatch Alarm	CloudWatch Events (EventBridge)
----------------	-------------------------	--

Output	Alert/Notification	Action trigger
--------	--------------------	----------------

👉 Real usage:

Alarm → triggers Event → invokes Lambda

6. Overlapping CIDR Issue

Problem:

- Two VPCs have same CIDR → cannot communicate

Solution:

- Change CIDR block
- Add secondary CIDR
- Re-design network

👉 This is a **very common architecture mistake**

7. EC2 vs S3

EC2	S3
Compute	Storage
Runs apps	Stores objects
OS-based	Serverless
Pay per usage	Pay per storage

👉 Simple line:

EC2 runs applications, S3 stores data.

8. Can EC2 be in Multiple Target Groups?

✓ Yes

- Same EC2 can serve:
 - /app1 → TG1
 - /app2 → TG2

👉 Used in microservices routing

9. EC2 Instance Types

- General Purpose → t, m
- Compute Optimized → c
- Memory Optimized → r
- Storage Optimized → i, d
- GPU/Accelerated → p, g

👉 Always mention **use case**, not just names.

10. S3 Storage Classes

- Standard
- Intelligent-Tiering
- Standard-IA
- One Zone-IA
- Glacier
- Glacier Deep Archive

👉 Interview tip:

Say *“cost optimization based on access pattern”*

11. Target Group Unhealthy – Causes

Common reasons:

- ❌ Wrong health check path

- ✗ Port mismatch
- ✗ App not running
- ✗ Security group blocking
- ✗ Timeout issues

👉 Always say:

“First I check health check config, then app logs, then SG/NACL.”

🧠 How You Should Answer in Interview

Instead of long explanation like video, use this structure:

👉 **1 line definition**

👉 **2–3 bullet points**

👉 **1 real example**

Example:

“VPC Endpoint allows private connectivity to AWS services without internet. I used it to connect EC2 to S3 securely and reduce NAT cost.”

🔥 AWS Scenario-Based Interview Questions (Strong Answers)

✅ 1. CI/CD Setup for Java Application (End-to-End)

Question:

Java app in Git → deploy to EC2 / Kubernetes. Explain steps.

✅ **Perfect Answer:**

“I design the pipeline in stages covering build, artifact, containerization, and deployment.”

🔹 **Flow:**

1. Source Stage

- Code from Git (GitHub/GitLab)

2. Build Stage

- Use Maven/Gradle
- Run:

mvn clean package

3. **Artifact Storage**

- Store JAR in artifact repo (Nexus/S3)

4. **Docker Build**

- Create Docker image
- Push to registry (ECR/Docker Hub)

5. **Deployment**

- **EC2** → SSH + deploy container
- **Kubernetes** → Helm / manifests

6. **CI/CD Tool**

- Jenkins / GitHub Actions / GitLab CI

7. **Extras (important to mention)**

- Code quality (SonarQube)
- Security scan
- Rollback strategy

📖 One-line summary:

“End-to-end pipeline from code commit to production using containerized deployment.”

✓ 2. **Helm vs Manifest (Kubernetes)**

Answer:

Helm	Manifest
Templating	Static YAML
Reusable	Hardcoded

Helm

Manifest

Easy for multi-env Difficult to manage

👉 Say this:

“I prefer Helm for real projects because it supports reusability and environment-based configurations.”

✓ 3. Dockerfile (Interview Expectation)

Minimum you should say:

- Base image
- Copy code
- Install dependencies
- Run application

👉 Example:

```
FROM openjdk:17
COPY target/app.jar app.jar
CMD ["java", "-jar", "app.jar"]
```

👉 Bonus:

- Multi-stage build (optimization)
-

✓ 4. Kubernetes Architecture

Master Components:

- API Server
- Scheduler
- Controller Manager
- etcd

Worker Node:

- Kubelet
- Kube-proxy
- Container Runtime

👉 One-liner:

“Master manages cluster, worker runs workloads.”

✓ 5. Expose Application (Nginx Pod)

Options:

- ClusterIP → internal
- NodePort → basic external
- LoadBalancer → production

👉 Best answer:

“For production, I use LoadBalancer or Ingress.”

✓ 6. Pod Communication Across Namespaces

Answer:

- By default → communication allowed
- Restrict using **Network Policies**

👉 Example:

“I define network policies to allow only required traffic between namespaces.”

✓ 7. Secret vs ConfigMap

Secret	ConfigMap
Sensitive data	Non-sensitive
Base64 encoded	Plain text

Secret	ConfigMap
--------	-----------

Passwords, keys Env configs

👉 Strong line:

“Secrets are used for credentials, ConfigMaps for configuration.”

✓ 8. Auto Scaling Parameters

- CPU utilization
- Memory (custom metrics)
- Request count (ALB)
- Queue size (SQS)

👉 Say:

“Scaling is based on metrics like CPU or request load.”

✓ 9. EC2 vs Lambda

EC2	Lambda
Server-based	Serverless
Runs continuously	Event-driven
Manual scaling	Auto scaling

👉 One-liner:

“EC2 is for long-running apps, Lambda for event-driven workloads.”

✓ 10. When to Use EC2 vs Lambda (Scenario)

Use Lambda when:

- Event-based (API, S3 trigger)
- Short execution

- No infra management

Use EC2 when:

- Long-running apps
- Custom OS/config needed
- Stateful apps

👉 Interview killer line:

“Lambda reduces operational overhead, EC2 gives full control.”

🔒 AWS Interview Questions (Refined & Correct Answers)

✅ 1. Resource-Based Policy vs Identity-Based Policy

⚠️ First correction:

It's **not** “user-based role” vs “resource-based role” — correct terms are:

✓ Identity-Based Policy

- Attached to:
 - Users / Groups / Roles
- Defines:
 - What actions identity can perform

👉 Example:

- IAM Role attached to EC2 → access S3
-

✓ Resource-Based Policy

- Attached to:
 - Resource (S3, Lambda, etc.)
- Defines:
 - Who can access the resource

👉 Example:

- S3 bucket policy allowing another account access
-

Interview line:

“Identity-based policies control what a user/role can do, while resource-based policies control who can access the resource.”

✓ 2. Securing VPC Peering Connections

Good answer structure:

- Use **non-overlapping CIDR**
- Restrict traffic using:
 - Security Groups
 - NACLs
- Update **route tables carefully**
- Enable:
 - VPC Flow Logs (for monitoring)

 Important:

- No transitive routing
 - No default encryption → use app-level encryption if needed
-

✓ 3. High Availability & Fault Tolerance (EC2)

Correct & strong answer:

- Deploy EC2 across **multiple Availability Zones**
- Use:
 - Auto Scaling Group
 - Elastic Load Balancer
- Store state externally (RDS/S3)

- Enable:
 - Health checks
 - CloudWatch monitoring
-

Perfect line:

“I distribute instances across multiple AZs using Auto Scaling and Load Balancer to ensure high availability.”

4. Pre-Signed URL in S3

Answer:

- Temporary access to private S3 object
- Time-limited (e.g., 5 minutes)
- No need to make bucket public

 Use case:

- Share file securely with external user
-

Interview line:

“Pre-signed URLs provide temporary, secure access to private S3 objects.”

5. VPC Flow Logs

Answer:

- Captures:
 - Network traffic metadata
- Applied at:
 - VPC / Subnet / ENI level
- Stored in:

- CloudWatch Logs
- S3

👉 Used for:

- Troubleshooting
 - Security auditing
-

✓ 6. Day-to-Day DevOps / Cloud Engineer Tasks

Give structured answer:

- Infrastructure provisioning (Terraform)
 - CI/CD pipeline management
 - Monitoring & alerting
 - Security & IAM management
 - Cost optimization
 - Troubleshooting production issues
-

🔥 Pro tip:

Avoid generic talk—add **1 real example**

✓ 7. Solutioning Experience

Answer:

“Yes, I have worked on designing end-to-end solutions including architecture design, POC implementation, and production rollout.”

👉 Mention:

- HA design
- CI/CD pipeline
- Kubernetes deployment

✓ 8. AWS Migration Tools

- AWS DMS → Database
- AWS MGN → Lift & Shift
- Snowball → Large data transfer

👉 Add:

- Planning via AWS Migration Hub
-

✓ 9. VPC Peering vs VPN (IMPORTANT FIX)

⚠ Your explanation had confusion. Here is correct:

✓ VPC Peering

- AWS ↔ AWS
 - Private connection
 - Same or different accounts
-

✓ VPN (Site-to-Site)

- On-prem ↔ AWS
 - Uses internet (encrypted)
-

🔗 Interview line:

“VPC peering connects AWS networks, while VPN connects on-prem to AWS.”

✓ 10. VPC Endpoint

- Private connection to AWS services
- No Internet/NAT needed

Types:

- Gateway → S3, DynamoDB
 - Interface → PrivateLink
-

✓ 11. Assessment Tools for Migration

- AWS Migration Hub
- AWS Application Discovery Service
- TCO Calculator

👉 These help in:

- Planning migration strategy
-

🚫 Key Mistakes You Should Avoid

✗ Saying:

- “User-based role”
- “VPC tunneling = cloud to cloud”

✓ Say instead:

- Identity-based policy
 - VPN / Peering / Transit Gateway correctly
-

🎯 How to Answer in Interview (Your Upgrade)

Instead of long explanation:

👉 Use this format:

Definition → Key points → Example

Example:

“VPC Flow Logs capture network traffic metadata. I use them for troubleshooting connectivity issues and auditing traffic.”

AWS Interview – Clean, Strong Answers

1. What is DHCP in AWS?

Answer:

DHCP (Dynamic Host Configuration Protocol) is used to automatically assign IP addresses and network configurations (DNS, gateway, subnet mask) to instances inside a VPC.

In AWS:

- Managed via **DHCP Option Sets**
- Controls:
 - DNS servers
 - Domain name
 - NTP servers

👉 Key point: *You don't manually assign IPs—AWS handles it via DHCP.*

2. Private Key vs Public Key

Answer:

- **Public Key** → Stored on EC2 instance (~/.ssh/authorized_keys)
- **Private Key** → Downloaded to your local system

Flow:

1. You initiate SSH using private key
2. AWS verifies it with stored public key
3. If matched → access granted

👉 Never share private key.

3. How SSH Works (Important Interview Question)

Answer (Short + Strong):

1. Client sends connection request

2. Uses private key to generate authentication
3. Server checks against public key
4. If matched → secure session established

👉 No password required → key-based authentication

4. What is SSM in AWS?

Answer:

SSM (Systems Manager) allows you to:

- Connect to EC2 **without SSH**
- Run commands remotely
- Store secrets (Parameter Store)

👉 Real-world use:

- No need for bastion host or key files
-

5. SSL Termination at Load Balancer

Answer:

- SSL is handled at Load Balancer (ALB)
- Uses AWS Certificate Manager (ACM)
- Traffic becomes HTTP after LB

👉 Benefits:

- Reduced load on EC2
 - Centralized certificate management
-

6. How to Secure EKS Cluster?

Answer:

- Enable **RBAC**

- Use **IAM roles for service accounts**
- Encrypt secrets
- Restrict network via **Security Groups + Network Policies**
- Enable **CloudWatch logging**

👉 Bonus: Use OIDC for fine-grained access

7. EC2 → S3 Access (Very Important)

Answer:

- Attach **IAM Role** to EC2
- Add S3 permissions in policy

Optional:

- Use **VPC Endpoint** for private access (no internet)
-

8. How to Make EC2 Highly Available?

Answer:

- Deploy across **multiple Availability Zones**
- Use:
 - **Auto Scaling Group**
 - **Application Load Balancer**
- Enable health checks

👉 This is a MUST answer for senior roles

9. EC2 in Private Subnet – How to Access?

Answer:

Two methods:

1. **Bastion Host (Jump Server)**

2. SSM (Recommended modern approach)

For internet access:

- Use **NAT Gateway**
-

10. Cooldown Period in Auto Scaling

Answer:

Cooldown period is the time Auto Scaling waits **after a scaling activity** before performing another action.

👉 Prevents:

- Frequent scaling (flapping)
 - Unstable infrastructure
-

11. IAM Role vs User-Based Access

Answer:

- **User-based** → Direct permissions to IAM user
- **Role-based** → Assigned to services (EC2, Lambda)

👉 Best practice: Always use **roles for services**

⚡ Interview Tip (Important)

Right now your content has:

- ✗ Repetition
- ✗ Mixed explanations
- ✗ No structure

In interviews, you must answer like this:

👉 **Definition → Use Case → Real Example**

Example:

"Auto Scaling ensures high availability by automatically adjusting the number of EC2 instances based on traffic. For example, in one project, we configured scaling based on CPU > 70%, which helped us handle sudden traffic spikes without downtime."

AWS Interview Questions (Part 2 – Clean Answers)

11. Auto Scaling Group – Parameters

Answer:

Auto Scaling automatically adjusts EC2 instances based on load.

Key parameters:

- Min / Max / Desired capacity
- Scaling policies:
 - Target tracking (e.g., CPU = 50%)
 - Step scaling (add/remove fixed instances)
 - Simple scaling (based on alarms)
- Health checks (EC2 / ELB)
- Cooldown period
- Launch template/config

 Example:

“We used CPU-based target tracking to scale from 2 to 10 instances during peak traffic.”

12. AWS Services in CI/CD Pipeline

Answer:

Typical stack:

- Source: GitHub / GitLab
- Build: Jenkins / CodeBuild
- Artifact: ECR / S3
- Deploy: EC2 / EKS
- IAM: Roles & permissions

 Optional:

- CodePipeline (end-to-end orchestration)
-

13. Handling High Traffic (E-commerce Scenario)

Answer:

- Use **Auto Scaling Group**
- Place instances behind **Application Load Balancer**
- Deploy across **multiple AZs**
- Enable **CloudWatch monitoring**

👉 Result: No downtime + automatic scaling

14. Auto Scaling + RDS Backend

Answer:

- Use **RDS endpoint (DNS)**, not IP
- All instances connect via same endpoint
- Enable:
 - Read replicas (for scaling reads)
 - Multi-AZ (for high availability)

👉 No need to reconfigure app when instances scale

15. Cross-Account S3 Access

Answer:

Two ways:

1. **Bucket Policy (most common)**
2. **IAM Role + AssumeRole (secure way)**

👉 Example:

- Account A bucket
- Account B role
- Add Role ARN in bucket policy

16. IAM Policy vs Role

Answer:

- **Policy** → Defines permissions
- **Role** → Entity that uses those permissions

🔗 Flow:

Policy → attached to → Role → assumed by → EC2 / User

17. STS (Security Token Service)

Answer:

STS allows **temporary access** to AWS resources.

Used for:

- Cross-account access
- AssumeRole
- Temporary credentials

🔗 More secure than long-term keys

18. Real-Time Issue (Important Question)

Best Answer Structure:

1. Problem
2. Root cause
3. Fix
4. Result

Example:

“We faced a production issue where database storage was full. Root cause was replication misconfiguration. We fixed replication settings and cleaned logs, which restored normal operations.”

👉 Keep it structured—this question decides selection.

19. What is a Subnet?

Answer:

A subnet is a **logical division of a VPC IP range**.

Used for:

- Isolation (public/private)
- Security control
- Routing

👉 Example:

- Public subnet → internet access
 - Private subnet → backend services
-

20. CIDR Calculation (Simple Way)

Instead of overcomplicating:

👉 Formula:

Total IPs = $2^{(32 - \text{subnet mask})}$

Example:

- /24 → $2^8 = 256$ IPs
 - Usable → 251 (AWS reserves 5)
-

21. ALB vs NLB (Very Important)

Feature	ALB	NLB
Layer	L7 (HTTP/HTTPS)	L4 (TCP/UDP)
Routing	Path-based	IP-based
Latency	Slightly higher	Ultra low

Feature	ALB	NLB
Use case	Web apps	High performance apps

👉 Simple line:

- ALB → Web apps
- NLB → High performance / real-time systems

⚡ What You're Doing Right (and Wrong)

✅ Good:

- Covering real interview questions
- Broad coverage (AWS, Docker, Terraform)

❌ Needs Fix:

- Answers are too long
- No structure
- Too much explanation, less impact

🌀 Golden Interview Formula

Always answer like this:

👉 **Definition → How it works → Example**

Example:

“Auto Scaling automatically adjusts EC2 instances based on load. It uses policies like CPU-based scaling. In my project, we scaled from 2 to 8 instances during peak hours.”

👤 1. NACL vs Security Group (Correct + Crisp Answer)

👉 Don't explain with stories in interviews. Say this:

Key differences:

- **Level**
 - NACL → Subnet level
 - Security Group → Instance level
- **State**
 - NACL → Stateless (inbound + outbound rules needed)
 - Security Group → Stateful (return traffic auto-allowed)
- **Rules**
 - NACL → Allow & Deny rules
 - Security Group → Only Allow rules
- **Evaluation**
 - NACL → Rule number order (priority matters)
 - Security Group → All rules evaluated together

👉 One-liner close:

“NACL is a coarse-grained firewall, Security Group is fine-grained.”

👉 2. EC2 → S3 Access (Best Practice Answer)

Your answer is mostly correct, but refine it:

👉 Say this:

“I attach an IAM Role with least-privilege S3 policy to the EC2 instance. This avoids hardcoding credentials and enables secure access.”

👉 Bonus (senior-level):

- Use **VPC Endpoint (Gateway)** to avoid internet
 - No need for NAT / IGW
-

👉 3. Trust Policy vs Permission Policy (Important Fix)

Your explanation is *partially correct but confusing*. Use this instead:

✓ **Permission Policy**

- Defines **what actions are allowed**
- Example: `s3:GetObject`

✓ **Trust Policy**

- Defines **who can assume the role**
- Example: EC2, Lambda, another AWS account

👉 Clean answer:

“Permission policy defines *what you can do*, trust policy defines *who can use the role*.”

👤 **4. Cross Account S3 Access (Strong Answer)**

👉 Say like this:

1. Create IAM Role in target account
2. Add trust policy for source account
3. Attach S3 access policy
4. Update S3 bucket policy with role ARN

👉 Bonus:

“We use STS AssumeRole for temporary credentials.”

👤 **5. VPC Endpoint (Fix + Clarity)**

Your concept is right but wording is messy.

👉 Clean version:

- Enables **private communication** with AWS services

- No Internet Gateway / NAT needed

Types:

- **Gateway Endpoint** → S3, DynamoDB
 - **Interface Endpoint** → PrivateLink (ENI-based)
-

6. API Gateway (Add Missing Depth)

You said basics—good. Add this:

 Proper answer:

- Managed service to expose REST/HTTP APIs
- Integrates with:
 - Lambda
 - EC2
 - ECS
 - DynamoDB

 Bonus:

- Supports throttling, auth, caching
-

7. Secrets Handling (Improve This – Important)

Your answer is incomplete.

 Better answer:

- Use:
 - AWS Secrets Manager
 - SSM Parameter Store
- Features:
 - Encryption (KMS)
 - Rotation (Lambda)
 - Fine-grained IAM access

👉 Senior touch:

“Avoid storing secrets in code, environment variables, or Terraform state.”

👤 8. S3 Storage Classes (Clean Version)

Instead of listing randomly:

- Standard
- Standard-IA
- One Zone-IA
- Glacier
- Glacier Deep Archive

👉 Add:

“Lifecycle policies automate movement to cheaper tiers.”

👤 9. Load Balancer (Fix Mistake ⚠️)

You made a small mistake:

👉 Correct:

- ALB → Layer 7 (HTTP/HTTPS)
- NLB → Layer 4 (TCP/UDP)

✗ You said NLB uses Layer 3 → Incorrect

👤 10. Strong Interview Answer Style (Very Important)

Right now your style is:

✗ Long

- ✗ Repetitive
- ✗ Story-like

👉 Use this format ALWAYS:

1. Definition (1 line)
 2. Key points (3–4 bullets)
 3. Real use case (1 line)
-

👤 Example (Perfect Answer)

Q: What is Auto Scaling?

“Auto Scaling automatically adjusts EC2 instances based on load.”

- Uses CloudWatch metrics (CPU, requests)
- Maintains desired capacity
- Supports dynamic scaling policies

👉 Use case:

“Used in e-commerce apps during traffic spikes.”

👤 1. NACL vs Security Group (Final Perfect Answer)

“Both are AWS firewalls, but operate at different layers.”

- **Security Group**
 - Instance level
 - Stateful
 - Only allow rules
- **NACL**
 - Subnet level
 - Stateless
 - Allow + Deny rules
 - Rule order matters

👉 Close strong:

“Security Group protects instances, NACL adds an extra subnet-level security layer.”

🔥 2. EC2 → S3 Access (3 Ways — Interview Ready)

You explained 3 ways, but refine like this:

✅ Best Practice (Recommended)

- Attach **IAM Role** to EC2
- No credentials needed

✅ Private Access

- Use **VPC Endpoint (Gateway)**
- No Internet/NAT required

⚠️ Not Recommended (but mention if asked)

- AWS CLI with access keys (hardcoded → bad practice)

👉 Say:

“IAM Role + VPC Endpoint is the most secure and production-grade approach.”

🔥 3. Trust Policy vs IAM Policy (Clean Answer)

“Trust policy defines *who can assume a role*, IAM policy defines *what actions are allowed*.”

Example:

- Trust → EC2 can assume role
- Policy → Allow s3:GetObject

👉 That’s it. Don’t over-explain.

🔥 4. Cross Account S3 Access (Senior-Level Answer)

- Create IAM Role in Account B

- Add trust policy for Account A
- Update S3 bucket policy
- Use **STS AssumeRole**

👉 Add:

“We use temporary credentials instead of static keys for security.”

🔥 5. VPC Endpoint (Strong Answer)

“VPC Endpoint enables private connectivity to AWS services without using the internet.”

Types:

- **Gateway** → S3, DynamoDB
- **Interface** → PrivateLink (ENI-based)

👉 Key benefit:

- No NAT cost
 - Secure internal traffic
-

🔥 6. S3 Storage Classes (Clean + Structured)

- Standard → Frequent access
- Standard-IA → Less frequent
- One Zone-IA → Cost optimized
- Intelligent-Tiering → Auto optimize
- Glacier → Archive
- Deep Archive → Long-term, cheapest

👉 Add:

“Lifecycle policies automate movement between these classes.”

7. Lifecycle Policy (Interview Answer)

“Used to transition or delete objects based on age.”

Actions:

- Transition (e.g., Standard → Glacier)
- Expiration (delete objects)

 Example:

“Logs moved to Glacier after 30 days.”

8. Load Balancer (ALB vs NLB — Correct + Sharp)

ALB

- Layer 7 (HTTP/HTTPS)
- Path-based routing
- Used for web apps, microservices

NLB

- Layer 4 (TCP/UDP)
- Ultra-low latency
- Used for real-time apps

 Strong close:

“ALB is for application logic, NLB is for performance.”

9. Auto Scaling Group (High-Quality Answer)

“Auto Scaling automatically adjusts EC2 instances based on demand.”

Key Parameters:

- Min / Max / Desired capacity
- Launch template
- Scaling policies

- Health checks

Scaling Types:

- Target tracking
- Step scaling
- Scheduled scaling

👉 Bonus:

“Helps in cost optimization and high availability.”

🔥 10. Cooldown Period (Simple Fix)

Your explanation is good, just simplify:

“Cooldown period is the time ASG waits after a scaling action before triggering another.”